Taller: Aritmética Digital y Representación Numérica

Lenin G. Falconí

2026-04-24

Contents

1 Representación entera en Computador Digital con Python **2**
1.1 Complemento a base disminuida 2
1.2 Complemento a la base 3
1.3 Preguntas y Problemas 6

2 Preguntas de Repaso (Capítulo 9) **6**

3 Problemas Propuestos (Capítulo 9) **7**

1 Representación entera en Computador Digital con Python

1.1 Complemento a base disminuida

Sea $r \geq 2$ la base, $n \in \mathbb{N}$, el número de dígitos[1], y $x \in \mathbb{Z}$ en **base 10** tal que

$$-(r^{n-1} - 1) \leq x \leq r^{n-1} - 1.$$

Definimos la representación en complemento a base disminuida $(r-1)$ como

$$C_{r-1}^{(n)}(x) = \begin{cases} x, & 0 \leq x \leq r^{n-1} - 1, \\ (r^n - 1) - |x|, & -(r^{n-1} - 1) \leq x < 0. \end{cases} \tag{1}$$

Entonces podemos escribir el siguiente código para $r = 2$

```
x = -15
n = 8
r = 2
limit_inferior = -(r**(n-1)-1)
limit_superior = r**(n-1)-1
# Verificar si el número esta en el rango
if  limit_inferior <= x <= limit_superior:
    # el complemento a 1 de X es:
    cmpl_1_dec = x
    if x  < 0:
        cmpl_1_dec = (r**n-1)-abs(x)
    # para expresar en binario de 8 bits
    cmpl_1_bin = format(cmpl_1_dec, '08b')
    cmpl_1_hex = format(cmpl_1_dec, '02X')
    print(f"La representación en complemento a {r-1} de {x} es:")
    print(f"{cmpl_1_bin} en base {r}")
    print(f"{cmpl_1_hex} en base 16")
else:
    print(f"{x} fuera de rango [{limit_inferior},{limit_superior}]")
```

```
La representación en complemento a 1 de -15 es:
11110000 en base 2
F0 en base 16
```

[1] bits cuando $r = 2$

También se puede obtener similar resultado restando el número x de un número formado por n dígitos de $r-1$. El siguiente código es para $r=2$

```
if  limit_inferior <= x <= limit_superior:
    # el complemento a 1 de X es:
    cmpl_1_dec = x
    if x  < 0:
        x_mod = abs(x)
        x_mod_bin = format(x_mod, '08b') # solo para r = 2
        print(f"|{x}| equivale en base {r} a :{x_mod_bin}")
        # genero una cadena con n digitos de valor r-1
        M = f"{r-1}"*n
        print(f"Restar {x_mod_bin} de {M}")
        cmpl_1_dec = int(M,r)-int(x_mod_bin,r)
    cmpl_1_bin = format(cmpl_1_dec, f'0{n}b')
    cmpl_1_hex = format(cmpl_1_dec, f'0{n//4}X')
    print(f"La representación en complemento a {r-1} de {x} es:")
    print(f"{cmpl_1_bin} en base {r}")
    print(f"{cmpl_1_hex} en base 16")
else:
    print(f"{x} fuera de rango [{limit_inferior},{limit_superior}]")
```

```
|-15| equivale en base 2 a :00001111
Restar 00001111 de 11111111
La representación en complemento a 1 de -15 es:
11110000 en base 2
F0 en base 16
```

1.2 Complemento a la base

Sea $r \geq 2$ la base, $n \in \mathbb{N}$ el número de dígitos y $x \in \mathbb{Z}$ tal que

$$-r^{n-1} \leq x \leq r^{n-1} - 1.$$

Definimos la representación en complemento a la base r (r-complement) como

$$C_r^{(n)}(x) = \begin{cases} x, & 0 \leq x \leq r^{n-1} - 1, \\ r^n - |x|, & -r^{n-1} \leq x < 0. \end{cases} \tag{2}$$

```
limit_inferior = -(r**(n-1))
limit_superior = r**(n-1) - 1

if limit_inferior <= x <= limit_superior:
    # Two's complement conversion
    cmpl_2_dec = x if x >= 0 else (r**n - abs(x))
    cmpl_2_bin = format(cmpl_2_dec, f'0{n}b')
    cmpl_2_hex = format(cmpl_2_dec, f'0{n//4}X')

    print(f"La representación en complemento a {r} de {x} es:")
    print(f"{cmpl_2_bin} en base {r}")
    print(f"{cmpl_2_hex} en base 16")
else:
    print(f"{x} fuera de rango [{limit_inferior},{limit_superior}]")
```

```
La representación en complemento a 2 de -15 es:
11110001 en base 2
F1 en base 16
```

Se puede programar la conversión inversa desde la cadena binaria en complemento a 2 a su equivalente decimal para verificar el resultado, usando

$$x = -b_{i-1} \times 2^{n-1} + \sum_{i=0}^{n-2} b_i \times 2^i$$

```
def complemento2_a_decimal(bin_str):
    n = len(bin_str)
    b = [int(bit) for bit in bin_str]
    b = b[::-1]
    # -b[n-1] * 2^(n-1) + sum(b[i] * 2^i for i in 0..n-2)
    result = -b[n-1] * (2 ** (n-1))
    for i in range(n-1):
        result += b[i] * (2 ** i)
    return result

# Example
x_ = complemento2_a_decimal(cmpl_2_bin)
print(f"El equivalente decimal de {cmpl_2_bin} en complemento a {r} es {x_}")
```

```
El equivalente decimal de 11110001 en complemento a 2 es -15
```

De manera similar podemos establecer la relación siguiente:

$$C_r^{(n)}(x) = C_{r-1}^{(n)}(x) + 1$$

```
if limit_inferior <= x <= limit_superior:
    # Handle zero and positive numbers (same as original)
    if x >= 0:
        cmpl_2_dec = x
        cmpl_2_bin = format(cmpl_2_dec, '08b')
        cmpl_2_hex = format(cmpl_2_dec, '02X')
    else:
        # Step 1: one's complement
        x_mod = abs(x)
        M = f"{r-1}"*n          # funciona si r = 2
        ones_complement_dec = int(M, r) - int(x_mod_bin, r)

        # Step 2: add 1 to get two's complement
        cmpl_2_dec = ones_complement_dec + 1

    cmpl_2_bin = format(cmpl_2_dec, f'0{n}b')
    cmpl_2_hex = format(cmpl_2_dec, f'0{n//4}X')

    print(f"La representación en complemento a {r} de {x} es:")
    print(f"{cmpl_2_bin} en base {r}")
    print(f"{cmpl_2_hex} en base 16")
else:
    print(f"{x} fuera de rango [{limit_inferior},{limit_superior}]")
```

```
La representación en complemento a 2 de -15 es:
11110001 en base 2
F1 en base 16
```

Un análisis similar puede hacerse para la representación **Signo Magnitud**. Pruebe a desarrollar programas que le permitan resolver los cálculos de los ejercicios. También, el código puede ser generalizado para trabajar sobre cualquier r. En estos ejemplos presentados, se ha escrito con $r = 2$ en mente.

1.3 Preguntas y Problemas

Se colocan algunos de los problemas del capítulo 9 de *Organización y Arquitectura de Computadores de W. Stallings*[1]. Puede modificar el archivo para hacer sus cálculos.

Sugerencia: Convertir el código explicado en funciones útiles que le permitan realizar los cálculos de manera adecuada.

2 Preguntas de Repaso (Capítulo 9)

Preguntas de Repaso: 9.1, 9.2, 9.3, 9.4, 9.8, 9.9

9.1 Explique brevemente las siguientes representaciones de enteros con signo:

- Signo-magnitud
- Complemento a dos
- Representación sesgada (biased)

Respuesta:

9.2 Explique cómo determinar si un número entero es negativo en cada representación:

- Signo-magnitud
- Complemento a dos
- Representación sesgada

Indique claramente qué patrón de bits se observa en cada caso.

Respuesta:

9.3 ¿En qué consiste la regla de extensión de signo para los números en complemento a dos?

Explique:

- Cómo se aplica al pasar de n a m bits (con m $>$ n).
- Por qué esta regla mantiene el valor numérico del entero.

Respuesta:

9.4 ¿Cómo se obtiene el opuesto de un entero (el negativo) en la representación de complemento a dos?

Describa el procedimiento paso a paso e ilustre con un ejemplo de 8 bits.

Respuesta:

9.8 ¿Cuáles son los cuatro elementos esenciales de un número en la notación de coma flotante?

Para cada elemento explique brevemente su función.

Respuesta:

9.9 ¿Qué ventaja supone utilizar la representación sesgada para la parte del exponente de un número en coma flotante?

Respuesta:

3 Problemas Propuestos (Capítulo 9)

Problemas: 9.1, 9.2, 9.7, 9.8, 9.10, 9.11, 9.23, 9.24, 9.28

9.1 Represente los siguientes números decimales con signo, usando **16 bits**:

- En **signo-magnitud**.
- En **complemento a dos**.

  1. +512
  2. -29

Indique sus resultados en binario y en hexadecimal.

Respuesta:

9.2 Represente en decimal los siguientes valores dados en **complemento a dos**:

1. 110011 (6 bits)
2. 010110 (6 bits)

Explique el procedimiento seguido en cada caso.

Respuesta:

9.7 En base r, el complemento de un número N de n dígitos se define como: $r^n - N$ para $N \neq 0$, y 0 para $N = 0$. Calcule el complemento a diez del número decimal 13250 **Respuesta:**

9.8 Calcule $72530 - 13250$ empleando aritmética en complemento a diez. Suponga reglas similares a las vistas para la aritmética en complemento a dos.

Respuesta:

9.10 Operaciones en complemento a dos (8 bits) Considere los números representados en **complemento a dos** con **8 bits** y realice los cálculos indicados. Señale si existe **desbordamiento**.

(a) 6 + 13 (b) -6 + 13 (c) 6 - 13 (d) -6 - 13

Escriba:

- La representación binaria de cada operando.
- El resultado binario (8 bits) y su valor decimal.

Respuesta:

9.11 Diferencias utilizando complemento a dos Calcule las siguientes diferencias utilizando aritmética de **complemento a dos**. Muestre el procedimiento de resta como suma con el complemento a dos del sustraendo.

(a) 111000 - 110011 (b) 1100100 - 101110 (c) 111100001111 - 110011110011 (d) 11000011 - 11101000

Indique:

- Resultado binario (con el mismo número de bits).
- Valor decimal correspondiente.

Respuesta:

9.23 Representación IEEE 754 de 32 bits Exprese en formato de coma flotante **IEEE 754 de 32 bits** los siguientes números decimales:

(a) -5 (b) -6 (c) -1,5 (d) 384 (e) 1/16 (f) -1/32

Para cada caso, indique:

- Signo, exponente (en binario y valor sesgado) y mantisa.
- Representación final en binario (32 bits) y en hexadecimal.

Respuesta:

9.24 Decodificación de IEEE 754 de 32 bits Los siguientes números emplean el formato de coma flotante **IEEE 754 de 32 bits**. Determine el número decimal equivalente en cada caso.

(a) 1 10000011 11000000000000000000000 (b) 0 01111110 10100000000000000000000 (c) 0 10000000 00000000000000000000000

Describa el procedimiento seguido:

- Interpretación del signo.
- Cálculo del exponente real.
- Reconstrucción de la mantisa.

Respuesta:

9.28 Exponentes sesgados Indique el **valor del sesgo** (`bias`) para los siguientes casos:

(a) Un exponente de **base 2** (`B = 2`) en un campo de **6 bits**. (b) Un exponente de **base 8** (`B = 8`) en un campo de **7 bits**.

Muestre cómo se obtiene el sesgo a partir del número de bits del exponente.

Respuesta:

9.41 Tabla de conversiones numéricas (8 bits) Considere una arquitectura de **8 bits**. Complete la siguiente tabla de conversiones numéricas para los valores decimales dados, expresando su representación en:

- **Complemento a 1** binario ($C_1(X)^{bin}$),
- **Complemento a 1** hexadecimal ($C_1(X)^{hex}$),
- **Complemento a 2** binario ($C_2(X)^{bin}$),
- **Complemento a 2** hexadecimal ($C_2(X)^{hex}$),
- **Signo-magnitud** binario ($SM(X)^{bin}$) y
- **Signo-magnitud** hexadecimal ($SM(X)^{hex}$).

X	$C_1(X)^{bin}$	$C_1(X)^{hex}$	$C_2(X)^{bin}$	$C_2(X)^{hex}$	$SM(X)^{bin}$	$SM(X)^{hex}$
+9						
-9						
+52						
-52						
+183						
-183						
+101						
-101						
+200						
-200						
+128						
-128						

Respuesta:

References

[1] William Stallings, Antonio Cañas Vargas, and Alberto Prieto Espinosa. *Organización y arquitectura de computadores*. Pearson Educación, 2006.